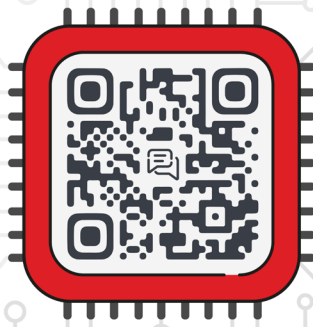




نظري

6

6

900 sp

كلية الهندسة المعلوماتية

السنة الأولى

التوابع 1

د. مدحت الصوص

د. فراس ضعيف



برمجة ا

RB Informatcs; 28/12/2025

محتوى مجاني غير مخصص للبيع التجاري

ظهرت فكرة التوابع في الجيل الثاني من أجيال لغات البرمجة وهو جيل اللغات الإجرائية

1. ما هي التوابع؟

تكمّن أهمية التوابع عندما نحتاج إلى إعادة استخدام مجموعة من التعليمات أكثر من مرة في أماكن مختلفة من البرنامج. فمثلاً عند حساب القاسم المشترك الأكبر لأعداد مختلفة عدة مرات داخل البرنامج، فإن تكرار كتابة نفس التعليمات يؤدي إلى زيادة حجم البرنامج وصعوبة قراءته وتعديله. وهنا يكون الحل الأمثل بتعريف تابع خارج تابع main، حيث نكتب فيه الخوارزمية مرة واحدة، ثم نستدعي هذا التابع عند الحاجة مع تغيير قيم الدخل.

يمكننا القول بأن التابع هو برنامج مصغر يملك:

- دخلاً يتمثل في وسطاء (Parameters) تمرّر له عند الاستدعاء.
- مجموعة تعليمات ينفذها على هذه القيم.
- خرجاً يتمثل في قيمة واحدة كحد أقصى يعيدها التابع (وفقاً لما تسمح به لغة C++).

المقصود بالدخل والخرج في التوابع ليس الإدخال من لوحة المفاتيح أو الإخراج إلى الشاشة، وإنما القيم التي يستقبلها التابع ويعيدها كنتيجة.

استخدام التوابع يساعد على:

- اختصار حجم البرنامج وتقليل تكرار الكود.
- تسهيل قراءة البرنامج وفهمه وصيانته.

2. طريقة تعريف التابع syntax:

{ (...., وسيط 2 ونوعه, وسيط 1 ونوعه) اسم التابع نوع القيمة التي سيعيدها التابع

مجموعة تعليمات

```
return □;
}
```

هناك نوعان من التوابع:

- 1- توابع تعيد قيمة عند انتهاء تنفيذها نوعها `int, float, string, ...`
- 2- توابع لا تعيد أي قيمة عند انتهاء تنفيذها `void` حيث تكتب `void` للدلالة على أن التابع لا يعيد أي قيمة

ملاحظات:

1. لا يمكن تعريف تابع داخل تابع آخر، بل يجب تعريف جميع التوابع بشكل مستقل عن بعضها البعض.
2. لا يمكن تسمية وسيطين بالاسم نفسه
3. يجب أن يحقق اسم التابع نفس قواعد تسمية المتحولات في لغة C++
4. يمكن استدعاء أي تابع من داخل تابع آخر بذكر اسمه وتزويد قيم الوسيطة إليه، ثم استخدام القيمة المعادة في الطباعة أو في عمليات حسابية أخرى.
5. في حال كان التابع مكون من سطر واحد يجب علينا وضع هذه الأقواس {} ولو كان لا يستقبل دخل فإن هذه الأقواس () ضرورية أيضاً سواء كان من نوع `void` أو غيره
6. عند تعريف وسيطة التابع يجب كتابة نوع كل وسيطة مثل `int x, int y` ولا يمكننا اختصار العبارة بكتابة `int x,y`

لإنهاء تنفيذ التابع يجب كتابة التعليمة `return` التي تقوم بـ:

- إنهاء تنفيذ التابع
- وتسمح بإعادة قيمة في حال كان التابع يعيد قيمة.

في حال كان التابع لا يعيد قيمة <code>void</code>	في حال كان التابع يعيد قيمة واحدة <code>int, float, ...</code>
كتابة <code>return</code> عند الانتهاء من تعليمات التابع امر اختياري وإن أردنا كتابتها نكتب <code>return</code> فقط دون قيمة داخلها	كتابة <code>return</code> عند الانتهاء من تعليمات التابع امر اجباري ونكتبها على الشكل <code>return</code> ؛ حيث نضع داخلها القيمة التي نريد إرجاعها

في نظام التشغيل عند الانتهاء من كتابة البرنامج وبعد الضغط على `run` فإن نظام التشغيل ينفذ التعليمات داخل تابع `main` فقط ولا ينفذ التعليمات ضمن بقية التوابع إذ أن التابع `main` يعتبر نقطة بداية تنفيذ البرنامج

ما الفائدة من كتابة التوابع إن كان نظام التشغيل لا ينفذ سوى تابع `main`؟

- نستدعي التوابع عند حاجتنا إليها داخل تابع `main`
- وأثناء عملية التنفيذ عند الوصول لاسم تابع ما
- تتم عملية البدء بتنفيذه وقراءة التعليمات بداخله

يتم الاستدعاء عن طريق ذكر اسم التابع كتعليمة منفصلة مع الوسيطة التي يأخذها (إن وجد)

مثال: لاستدعاء تابع `sqrt` تابع الجذر نكتب داخل ال `main`: `sqrt(parameter);`

مثال: اكتب برنامج يستخدم فكرة التتابع لإعادة ناتج جميع عددين:

```
#include <iostream>
using namespace std;
int sum (int x, int y) {
    int z = x + y;
    return z;
}
int main () {
    int a, b;
    cin >> a >> b;
    cout << sum (a, b);
    return 0;
}
```



المثال السابق يعيد قيمة int للتابع

كيف يكون شكل التابع من نوع void؟

مثال: اكتب برنامج لتابع يقوم بطباعة Hello world

```
#include <iostream>
using namespace std;
void Hello () {
    cout << "Hello world" << endl;
    return;
}
int main () {
    Hello ();
    return 0;
}
```



إذا لماذا نستخدم التابع من غير نوع void ونحن يمكننا كتابة:

```
void sum (int a, int b) {
    int z;
    z = a + b;
    cout << z << endl;
}
```

في هذه الحالة التابع سيقوم التابع بطباعة المجموع على الشاشة دائماً

ولكن إذا احتجنا إلى حساب المتوسط، فنحن نحتاج إلى قيمة المجموع نفسها (وليس طباعتها فقط) لكي نقسمها على عدد العناصر.

لذلك فإن التابع من نوع void لن يفيدنا هنا، لأنه لا يعيد أي قيمة، وبالتالي لا يمكن استخدام نتيجته في عمليات أخرى

تمرير الوسطاء:

1. تمرير الوسطاء بالقيمة: pass by value

عند تمرير الوسطاء إلى التابع فإننا نمرر قيمة المتحولات إلى التابع ولا نمرر المتحولات نفسها (يأخذ التابع نسخة عنها) وبالتالي عند تمرير الوسطاء بالقيمة إلى تابع ما فذلك يمنع حدوث أي تغيير على المتحول الأصل طريقة تعريف الوسطاء التي يتم التمرير عبرها بالقيمة:

```
int function (int x, int y) {}
```

2. تمرير الوسطاء بالعنوان: pass by address

عند تمرير الوسطاء إلى التابع فإننا لا نمرر القيمة فقط، بل نمرر المتحولات نفسها وبالتالي فإن أي تغيير على المتحول الوسيط في التابع فسيحدث أيضاً على المتحول الأصل طريقة تعريف الوسطاء التي يتم التمرير عبرها بالعنوان:

```
int function (int& x, int& y) {}
```

الرمز & يستخدم هنا لتمرير المتحول بالعنوان (reference) ولا علاقة له بالرمز && المستخدم كعامل منطقي بمعنى AND

مثال شامل:

اكتب برنامج فيه تابع يقوم بطباعة قيمة ما ثم ينتقل إلى السطر التالي وتابع آخر يقوم بجمع 3 أعداد

```
#include <iostream>
using namespace std;
void print (int x) {
    cout << x << endl;
}
int sum (int x, int y, int z) {
    int k = x + y + z;
    return k;
}
int main () {
    int x, y, z;
    cin >> x >> y >> z;
    print (sum(x, y, z));
    print (sum(x + 1, y + 1, z + 1));
    return 0;
}
```



1. يتم حذف قيمة متحولات التابع من الذاكرة RAM بعد انتهاء تنفيذه
 2. يمكن للمتحولات داخل تابع ما أن تكون مطابقة بالاسم لمتحولات تابع آخر لكن باختلاف القيم بداخلها كما في المثال السابق.
 3. في حال كان التابع يستقبل 5 أعداد كحد أقصى، وللمستخدم حرية اختيار عدد القيم التي يريد إرسالها، فإننا نضع قيمة ابتدائية لوسطاء التابع (عدا الوسطاء الإجبارية): صفراً في حالة التابع الخاص بالجمع، وواحداً في حالة التابع الخاص بالضرب.
- مثال لتوضيح الملاحظة 3:

```
int add (int x, int y, int z = 0) {
    return x + y + z;
}
```

هنا أصبح إدخال z اختيارياً، أي إذا أدخله المستخدم سيأخذ القيمة المرسله وفي حال لم يدخله سيأخذ القيمة 0

فائدة RBCs

نستطيع الآن التحدث عن التوابع الجاهزة الموجودة في مكتبة `cmath` او `math.h` يتطلب استدعاء هذه التوابع فقط تضمين المكتبة واستدعاء التابع دون الحاجة للتعريف عنها

تابع الجذر التربيعي (`sqrt()`):

```
#include <iostream>
#include <cmath>
using namespace std;
int main () {
    int x = 25;
    cout << sqrt(x);
    return 0;
}
```

سيكون الخرج: 5

تابع التقريب (بالتدوير) لأقرب عدد صحيح (`round()`):

```
#include <iostream>
#include <cmath>
using namespace std;
int main () {
    float x = 2.5;
    cout << round(x);
    return 0;
}
```

سيكون الخرج: 3

تابع القوة (,) :pow

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int x = 2;
    int y = 3;
    cout << pow(2, 3);
}
```

سيكون الخرج: $2^3 = 8$

تابع لتقريب الرقم للرقم الأعلى :ceil ()

أيًا كان الرقم الذي يلي الفاصلة (عدا 0) يتم التقريب للعدد الأكبر

```
ceil (2.4);
ceil (2.0);
```

output: 3

output: 2

تابع لتقريب الرقم للرقم الأدنى :floor ()

أيًا كان الرقم الذي يلي الفاصلة (عدا 0) يتم التقريب للعدد الأصغر

```
floor (2.4);
floor (2.0);
```

output: 2

output: 2

تابع القيمة المطلقة :abs ()

```
abs (-10);
abs (10);
```

output: 10

output: 10



Stay curios. Keep coding.